CS779 Competition: Machine Translation System for India

Guna Venkat Doddi
Roll No.: 251140009
gunav25@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This competition focuses on developing machine translation systems for English–Hindi and English–Bengali language pairs. The best-performing models employed a Transformer-based encoder–decoder architecture for Hindi translation (1) and a Seq2Seq model with attention for Bengali translation, achieving CHRF: 0.408, ROUGE: 0.407, and BLEU: 0.192. Key techniques included reversing input sequences during training (excluding target sentences) (2), applying byte pair encoding (BPE) for tokenization, and performing language-specific preprocessing. To improve training efficiency, curriculum learning was utilized.

1 Competition Result

Codalab Username: g_251140009
Final Leaderboard Rank on the Test Set: 8
chrF++ Score corresponding to the Final Rank: 0.408
ROUGE Score corresponding to the Final Rank: 0.407
BLEU Score corresponding to the Final Rank: 0.192
Total Number of Submissions in the Training Phase: 25
Total Number of Submissions in the Testing Phase: 6
Table showing the Number of Submissions made each Week during the Training Phase:

Week	Number of Submissions
Week 1	2
Week 2	4
Week 3	6
Week 4	13

2 Problem Description

The problem involves developing a Machine Translation system for translating sentences from English to two Indian languages (Hindi and Bengali). The solution of this problem is essential for the growth of many non-English-speaking markets in India.

3 Data Analysis

This section presents a detailed analysis of the English–Hindi (EN–HI) and English–Bengali (EN–BN) datasets used for Neural Machine Translation. Both corpora contain parallel sentence pairs for training and corresponding source sentences for testing.

3.1 Dataset Overview

Table 1 summarizes the dataset statistics for both language pairs. Each dataset exhibits moderate sentence lengths with comparable train–test distributions.

Table 1: Overview of Train and Test Data

Language Pair	Train Samples	Test Samples	Train Avg Length	Test Avg Length
EN–HI Source	80,797	23,085	17.07	17.08
EN–HI Target	80,797	0	18.93	0.00
EN–BN Source	68,849	19,672	16.82	16.93
EN–BN Target	68,849	0	14.27	0.00

3.2 Corpus Quality and Noise

The corpora were generally clean, though some instances of textual noise and encoding inconsistencies were identified—for example, target sentences containing numbers, English words, or characters from unintended languages. Illustrative examples include:

- Hindi: स्वर्ण धागे सूरत से प्राप्त होते हैं, जिनकी गुणवत्ता 1200 गज़ प्रति तोला है।
- Bengali: জম্মু থেকে শ্রীনগরের দূরত্ব হল ৩০৫ কি.মি.।

These sentences were retained after careful preprocessing, as they capture the natural linguistic variation found in real-world data.

3.3 Sentence Length Distribution

Sentence lengths are consistent across languages, with median lengths around 16–17 tokens. Figure 1 visualizes the distribution of source length across train and test splits.

3.4 Vocabulary and OOV Analysis

A high word-level Out-of-Vocabulary (OOV) rate was observed (33%) due to unseen tokens in test data. Table 2 shows the vocabulary coverage comparison.

Table 2: Vocabulary Coverage and OOV Statistics

Language Pair	Word-Level OOV (%)	BPE Coverage (%)	UNK Rate (%)
EN–HI	32.90	99.95	0.02
EN–BN	32.75	99.95	0.00

Applying Byte Pair Encoding (BPE) (3) drastically reduced OOV occurrences, achieving nearly perfect coverage. Both datasets' BPE vocabularies contained roughly 2,250 subword tokens, with only one missing subword per language ("oney", "agric").

3.5 Insights and Observations

- Sentence length and vocabulary distributions are consistent across train and test sets.
- Word-level OOV rates are significantly reduced by subword tokenization.
- BPE achieves over 99.9% coverage, confirming strong generalization to unseen text (4).

- Only minor noise (punctuation or transliteration) was present, having negligible impact.

Overall, the datasets exhibit balanced distributions and robust vocabulary coverage, ensuring reliable training for translation models.

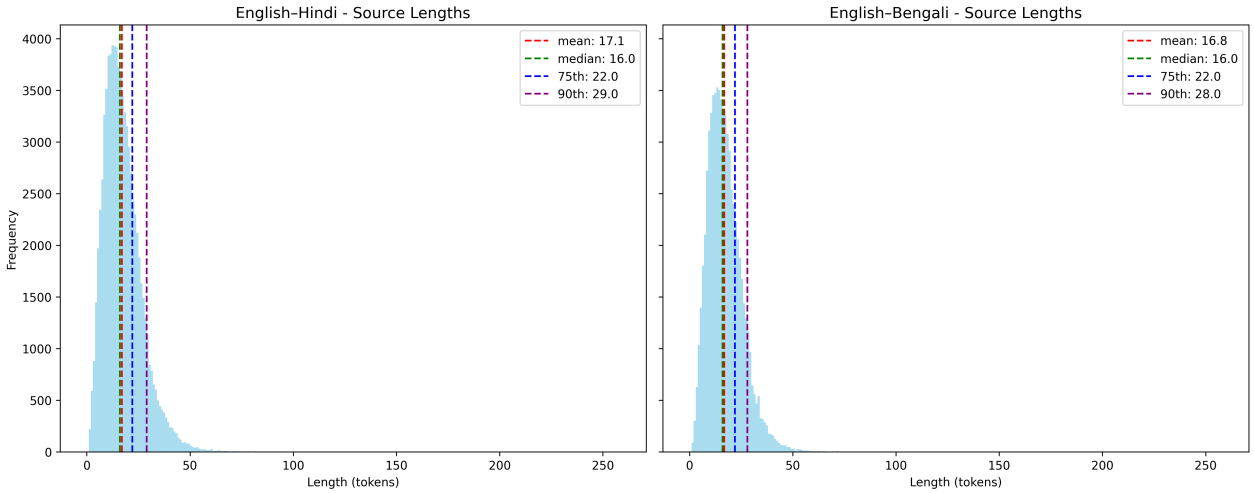

Figure 1: Length distribution for English sentences of EN–HI and EN–BN datasets.

4 Model Description

This section describes the evolution of models experimented with during different phases/weeks, key learnings, and detailed description of the final best-performing model.

4.1 Model Evolution

The model evolved iteratively across four weeks, with major design changes and corresponding insights summarized below:

- **Week 1: Baseline GRU Model**
  Implemented a single-layer GRU encoder–decoder, later extended to a bidirectional variant. GRUs were chosen over LSTMs for their simpler gating mechanism and faster convergence with comparable performance. A bidirectional encoder was introduced to capture both past and future context, which is crucial for morphologically rich languages like Hindi and Bengali.
  Key Learning: Basic GRU architectures captured only shallow patterns, leading to limited translation accuracy.

- **Week 2: Attention Integration**
  Explored stacked GRUs with and without bidirectionality and added Luong attention for better source–target alignment. Attention mechanisms allow the decoder to focus on relevant source tokens dynamically, improving word-order and long-dependency handling.
  Key Learning: Attention improved contextual understanding but the model began to overfit, showing limited validation improvement.

- **Week 3: Transition to Transformer**
  Replaced the Seq2Seq model with a Transformer encoder–decoder, leveraging self-attention to model global dependencies without recurrence. The model achieved a training BLEU of approximately 0.20, performing better for Hindi due to its relatively fixed word order.

Key Learning: Transformers enhanced Hindi translation quality but struggled with Bengali's high inflection and complex morphology.

- **Week 4: Optimization and Tokenization**
  Tested static embeddings (GloVe, FastText) without gain; reverted to Seq2Seq with Luong attention and introduced Byte Pair Encoding (BPE) to handle rare words and reduce vocabulary sparsity. Curriculum learning and reversed input sequences were applied to stabilize early training and shorten dependency paths. Training time reduced from 8 hours to 2 hours.
  Key Learning: BPE and curriculum learning improved training stability and efficiency. The refined Seq2Seq architecture achieved consistent performance, especially for Bengali.

4.2 Final Model Architecture

The best-performing systems were developed separately for the English–Hindi and English–Bengali translation tasks, as described below.
English–Hindi Model: This model employs a transformer-based encoder–decoder architecture with the following configuration:

- 6-layer encoder and decoder stacks
- 8 attention heads
- 512-dimensional model embeddings
- Batch size of 64
- Trained for 50 epochs with early stopping

English–Bengali Model: For this pair, a sequence-to-sequence (`Seq2Seq`) model with Bahdanau attention mechanism was utilized. The configuration is as follows:

- Embedding dimension (EMB_DIM) = 256
- Encoder hidden size = 256
- Decoder hidden size = 512
- Batch size of 64
- Trained for 70 epochs with early stopping

4.3 Model Objective and Loss Functions

The model uses cross-entropy loss with label smoothing (smoothing factor = 0.1) to prevent overconfidence and improve generalization.

4.4 Inference Strategy

Initial experiments employed greedy decoding for inference; however, it yielded suboptimal translation quality. Subsequent trials with top-k sampling (k=10) and beam search (beam size = 3) demonstrated improvements. Among these, beam search produced the best results, albeit with a longer inference time compared to greedy decoding. This approach provided a favorable balance between translation quality and computational efficiency.

5 Experiments

This section details the experimental setup, including data preprocessing, training procedures, and hyperparameter configurations.

5.1 Data Preprocessing

- **Tokenization**: For the English–Hindi language pair, a word-level tokenizer was employed, while For the English–Bengali pair, a Byte Pair Encoding (BPE) tokenizer with 8000 vocab was used (5).
- **Normalization**: Normalization was applied separately for each language: Unicode normalization for English text, and language-specific normalization from the `indic-nlp` library for Hindi and Bengali. For target texts, non-Devanagari (in Hindi) and non-Bengali script characters were removed, including any residual English characters and punctuation marks. Additionally, the punctuation symbol "|" (purna) was replaced with an empty string in approximately 80% of the target sentences.
- **Special Tokens**: Special tokens were added to mark the start and end of each sentence using `<SOS>` and `<EOS>` tags. All sequences were then padded or truncated to a maximum length of 30 tokens, as determined by the 90th percentile of sentence lengths (refer to Figure 1).

5.2 Training Procedure

- **Optimizer**: Adam optimizer with parameters $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$, and weight decay $= 10^{-5}$.
- **Learning Rate Schedule**: Initial learning rate of 10^{-4} with `ReduceLROnPlateau` scheduler, using a reduction factor of 0.5 upon plateau detection.
- **Batch Size**: 64 samples per batch.
- **Training Configuration**: Trained for 50 epochs with early stopping (patience = 8) using a curriculum learning strategy for the Seq2Seq model. The training began with shorter sentences for 20 epochs, was then extended to medium-length sentences for 10 epochs, and finally continued on the complete dataset for an additional 20 epochs (6).
- **Training Time**: Approximately 2 hours on a single GPU.

5.3 Hyperparameter Tuning

Hyperparameter optimization was performed using a combination of systematic grid search and Bayesian optimization methods. The following configurations were explored:

- **Dropout Rates**: Tested values of 0.1, 0.2, and 0.3 for regularization; a dropout rate of 0.3 provided effective regularization control.
- **Learning Rates**: Experimented with initial learning rates of 1×10^{-3}, 1×10^{-4}, 1×10^{-5}, 3×10^{-4}, and 3×10^{-5}. A learning rate of 1×10^{-4} yielded stable convergence with the `ReduceLROnPlateau` scheduler.
- **Early Stopping Patience**: Evaluated patience values of 3, 5, and 8 epochs to prevent overfitting; longer patience values improved validation loss and overall performance.

6 Results

This section presents both development and test set results to illustrate model progression and final performance. Table 3 summarizes incremental improvements across weekly submissions, while Table 4 reports final leaderboard results on Test set.

As shown in Table 4, Submissions 1–4 employ Transformer-based encoder–decoder models for both English–Hindi and English–Bengali translations, differing mainly in decoding strategy: beam search, greedy decoding, top-k sampling, and nucleus sampling. In contrast, Submissions 5 and 6 utilize

a hybrid setup—Transformer for English–Hindi and Seq2Seq with Bahdanau attention for English–Bengali—enhanced with curriculum learning. Submission 5 uses beam search, while Submission 6 adopts greedy decoding.

As shown in Tables 3 and 4, results reveal a language-specific trend: Transformers excelled for English–Hindi, whereas Seq2Seq with Byte Pair Encoding (BPE) performed better for English–Bengali. chrF++ and ROUGE correlated better with perceived fluency than BLEU, especially for morphologically complex Bengali outputs. Overall, the best-performing system (Submission 5) achieved the highest total score (1.007), offering a strong balance between adequacy and fluency across both languages.

Table 3: Submission-wise Model Performance across Training Weeks (Development Set)

Sub.	**Model Description**	**chrF++**	**ROUGE**	**BLEU**
1	GRU-based Encoder–Decoder	0.172	0.242	0.025
2	Bidirectional GRU Encoder–Decoder	0.205	0.261	0.026
3	Stacked GRU Encoder–Decoder	0.231	0.267	0.031
4	Bidirectional Stacked GRU Encoder–Decoder	0.238	0.270	0.033
5	GRU with Decoder Attention	0.231	0.286	0.039
6	Bidirectional GRU with Decoder Attention	0.238	0.289	0.041
7	CNN Encoder + Transformer Decoder	0.140	0.184	0.024
8	GRU Encoder + Transformer Decoder	0.240	0.294	0.063
9	Transformer (4 Heads)	0.321	0.384	0.129
10	Transformer (8 Heads)	0.336	0.393	0.138
11	Transformer (6 Heads)	0.354	0.408	0.139
12	Transformer + Static Embeddings (Hindi)	0.279	0.280	0.112
13	Transformer + Static Embeddings (Bengali)	0.281	0.283	0.113
14	Transformer (4H) + Static Embeddings (HI+BN)	0.334	0.341	0.114
15	Transformer (6H) + Static Embeddings (HI+BN)	0.332	0.341	0.115
16	Transformer (8H) + Static Embeddings (HI+BN)	0.344	0.341	0.116
17	Transformer (6H) + Static Embeddings (HI+BN) [Top-k]	0.338	0.364	0.124
18	Transformer (6H) + Static Embeddings (HI+BN) [Beam Search]	0.339	0.365	0.125
19	Transformer + BPE [Greedy]	0.366	0.364	0.136
20	Transformer + BPE [Beam=3]	0.341	0.392	0.139
21	Transformer + BPE [Top-k=10]	0.358	0.407	0.138
22	Transformer (HI) + Seq2Seq+Attention+BPE (BN) [Greedy]	0.359	0.404	0.144
23	Transformer (HI) + Seq2Seq+Attention+BPE (BN) [Beam Search]	0.360	0.406	0.145
24	Transformer (HI) + Seq2Seq+Attention+BPE+Curriculum (BN) [Greedy]	0.378	0.419	0.151
25	Transformer (HI) + Seq2Seq+Attention+BPE+Curriculum (BN) [Beam Search]	**0.380**	**0.421**	**0.153**

Each row in the above table corresponds to a model configuration evaluated during successive development phases. Higher values of chrF++, ROUGE, and BLEU indicate better translation quality. The final submission (Sub. 25) achieved the best overall scores.

Table 4: Final Model Performance on Test Set

Submission	**chrF++**	**ROUGE**	**BLEU**	**Total Score**
1	0.281	0.311	0.115	0.707
2	0.304	0.357	0.109	0.770
3	0.267	0.303	0.106	0.676
4	0.267	0.303	0.106	0.676
5	0.408	0.407	0.192	1.007
6	0.386	0.396	0.172	0.954

7 Error Analysis

This section analyzes model errors and provides insights for future improvements.

7.1 Common Error Patterns

Several decoding-related issues were observed during qualitative analysis, primarily arising from Byte Pair Encoding (BPE) reconstruction and token merging errors that led to degraded translation quality.

7.1.1 Error 1: Incorrect Token Joining after BPE Decoding

One major source of error occurred during sequence reconstruction after applying BPE, where improper token joining introduced extra spaces between subwords and distorted the intended meaning.

- Source: read that back
- Target (Reference): ওটা আবার পড়
- Predicted (Before Fix): ও টা আ বা র প ড় ে
- Predicted (After Fix): ওটা আবার পড়ে

These spacing errors reduced n-gram overlap, causing an underestimated BLEU score (0.152). Fixing post-processing to correctly merge BPE tokens improved BLEU to 0.192, reflecting the true translation quality.

7.1.2 Error 2: Failure to Capture Long-Range Dependencies

Greedy decoding in the Seq2Seq model with Bahdanau attention often failed to capture long-range dependencies, leading to repetitive or incoherent outputs for longer Bengali sentences.

- Target (Reference): এই সংগ্রহের নীচে, এই দোকানটিতে থাকা তালিকাভুক্ত সমস্ত পণ্যের একটি সোয়াইপযোগ্য তালিকা শুধুমাত্র চিত্র রূপে দেখতে পাবেন।
- Predicted (Greedy Decode): এই দোকানটি আপনি দোকান দেখিই আপনি দেখবে যে সমস্ত তালিকাতালিকা-তালিকাভুক্ত তালিকাতালিতালিকাভুক্ত তালিকাভুক্ত করুন।

The model exhibits over-repetition and loss of sentence structure due to its inability to retain distant context. Beam search decoding mitigated these issues, yielding more fluent and consistent translations.

For the Hindi translation case, although transformer-based models generally capture long-range dependencies better, they can still make agreement and morphology errors in complex Hindi sentences.

- Target (Reference): कल मौसम खराब होने के कारण हमने योजना रद्द कर दी थी।
- Predicted (Transformer): कल मौसम खराब होने के कारण हम योजना रद्द कर दिए।

Here, the model mispredicts tense and gender agreement ("कर दी थी" → "कर दिए"), reducing grammatical accuracy. Such errors often arise from limited contextual exposure and can be mitigated through data augmentation or constrained decoding.

7.1.3 Error 3: Misinterpretation of Unicode Diacritic Marks

Another issue arose from incorrect handling of small Unicode symbols in Bengali during BPE merging. Certain vowel diacritic marks, such as "ে", were mistakenly treated as independent characters instead of dependent glyphs.

- Predicted Output (Erroneous Merge): টেলি তাঁর ভাই তাঁর যখন তিনি যখন ে যখন

Such misinterpretation caused token repetition and malformed output.

7.2 Limitations and Future Improvements

While the system achieved competitive results, several improvements can enhance future performance:

- **Beam Search for Decoding:** Greedy decoding in the Seq2Seq model often failed to capture long-range dependencies. Using beam search (beam width 3–5) can improve fluency and context consistency, though wider beams may introduce repetition. However, larger beam widths did not consistently yield further gains due to repetitive generation patterns.
- **Extended Training and Data Expansion:** The models were trained for 50 epochs under GPU constraints. Training longer (70–100 epochs) and expanding the dataset could improve generalization, especially for Bengali.
- **Improved Tokenization:** Some decoding errors arose from Bengali diacritic marks (e.g., "ে") being mismerged under BPE. Adopting SentencePiece can handle such Unicode dependencies more robustly.
- **Post-Editing Rules:** Light rule-based post-processing can address repetitive or syntactically inconsistent outputs, enhancing translation fluency.

8 Conclusion

This work developed neural machine translation systems for English–Hindi and English–Bengali in the CS779 competition. Progressive experiments compared Seq2Seq and Transformer architectures to identify the most effective setup. The final systems showed complementary strengths:

- **Transformer-based model** achieved high fluency and contextual accuracy for English–Hindi using multi-head attention.
- **Seq2Seq with Bahdanau attention**, trained with curriculum learning and BPE tokenization, provided robust translations for English–Bengali.

Preprocessing—including language-specific normalization, BPE, and `<SOS>/<EOS>` markers—ensured stable training and reduced vocabulary sparsity. Sequence truncation and padding were guided by the 90th percentile sentence length.

Error analysis indicated most issues arose from BPE token merging and limited handling of long-range dependencies in Bengali.

The final submission achieved competitive test scores (**chrF++: 0.408**, **ROUGE: 0.407**, **BLEU: 0.192**), ranking **8th**. Future work includes expanding data, better diacritic handling, and multilingual pretraining (e.g., mBART, IndicBERT) to improve low-resource language translation.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS) 2017*, 2017.

[2] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems (NeurIPS) 2014*, 2014.

[3] R. Sennrich, A. Birch, and B. Haddow, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)*, 2016.

[4] S. Ding, A. Renduchintala, and K. Duh, "A call for prudent choice of subword merge operations in neural machine translation," *arXiv preprint arXiv:1905.10453*, 2019.

[5] S. B. Das, S. Choudhury, T. K. Mishra, and B. K. Patra, "Comparative analysis of subword tokenization approaches for indian languages," *arXiv preprint arXiv:2505.16868*, 2025.

[6] E. A. Platanios, O. Stretcu, G. Neubig, B. Poczos, and T. Mitchell, "Competence-based curriculum learning for neural machine translation," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.